

Lab 11: Równania różniczkowe zwyczajne (Ordinary Differential Equations)

Vladyslav Humeniuk

1 Introduction

Celem niniejszego laboratorium było rozwiązywanie równań różniczkowych zwyczajnych (*Ordinary Differential Equations*, ODEs) przy użyciu metod numerycznych udostępnianych przez bibliotekę GNU Scientific Library (GSL) [1]. Rozwiązania numeryczne są niezbędne, gdy uzyskanie rozwiązań analitycznych jest trudne lub niemożliwe; polegają one na zastąpieniu równań różniczkowych przybliżeniami algebraicznymi obliczanymi w skończonych krokach (*finite steps*) [2].

Sprawozdanie podzielono na dwie główne części. Część A bada oscylator van der Pola, nieliniowe równanie różniczkowe drugiego rzędu, skupiając się na różnicach pomiędzy solverami o stałym kroku (*fixed-step*) i adaptacyjnym kroku (*adaptive-step*). Część B dotyczy rozwiązania problemu początkowego (*initial value problem*) pierwszego rzędu w celu porównania dokładności i wydajności metody Rungego-Kutty 4. rzędu ze standardową metodą Eulera.

2 Methodology

2.1 Część A: Oscylator van der Pola (The van der Pol Oscillator)

Równanie van der Pola ma postać:

$$u''(t) + \mu u'(t)(u(t)^2 - 1) + u(t) = 0$$

Aby rozwiązać to za pomocą GSL, równanie musi zostać sprowadzone do układu równań pierwszego rzędu poprzez wprowadzenie nowej zmiennej $v = u'(t)$ [2]:

$$u' = v$$

$$v' = -u + \mu v(1 - u^2)$$

Przeanalizowano trzy różne implementacje oparte na GSL:

1. **ode-ex1.c**: Wykorzystuje wysokopoziomowy *driver* z adaptacyjną metodą Rungego-Kutty Prince'a-Dormanda (8,9) (**rk8pd**). Oblicza rozwiązanie dynamicznie, ale wymusza wyjście (*output*) w dokładnie 100 równomiernie rozłożonych odstępach.

2. **ode-ex2.c**: Stosuje niskopoziomowy *stepper* (`gsl_odeiv2_evolve`) z metodą `rk8pd`. Wypisuje rozwiązanie w każdym pojedynczym, wewnętrznym kroku adaptacyjnym wykonanym przez algorytm.
3. **ode-ex3.c**: Wykorzystuje wysokopoziomowy *driver* ze ścisłą metodą Rungego-Kutty 4. rzędu o stałym kroku (*fixed-step*, `rk4`), wykonując 1000 wewnętrznych kroków na przedział, aby wypisać 100 równomiernie rozłożonych punktów.

2.2 Część B: Porównanie dla równania różniczkowego pierwszego rzędu

Docelowy problem początkowy (*initial value problem*, $y(0) = 0$) to:

$$y' + y \cos(x) = \sin(x) \cos(x)$$

Zapisane w postaci standardowej $y' = f(x, y)$:

$$y' = \sin(x) \cos(x) - y \cos(x)$$

Równanie zostało rozwiązane w przedziale $[0, 15]$ przy użyciu dwóch metod numerycznych:

1. Metoda Rungego-Kutty 4. rzędu (poprzez GSL `gsl_odeiv2_step_rk4`).
2. Metoda Eulera (implementacja własna).

Wyniki numeryczne zostały porównane z dokładnym rozwiązaniem analitycznym:

$$y(x) = e^{-\sin(x)} + \sin(x) - 1$$

3 Results

3.1 Część A: Strategie solverów dla równania van der Pola

Rysunek 1 ilustruje rozkład kroków (*step distribution*) dla trzech konfiguracji solvera.

Jak widać na wykresach nakładających się (*overlay plots*, Rysunki 2, 3 i 4), wszystkie trzy metody z powodzeniem śledzą tę samą krzywą oscylacyjną. Jednakże wewnętrzne mechanizmy różnią się drastycznie:

- **Przykład 1 i 3** dają idealnie równomiernie rozłożone wyniki. Przykład 1 osiąga to poprzez maskowanie złożonych adaptacyjnych mikrokroków, podczas gdy Przykład 3 osiąga to poprzez siłowe (*brute-force*) mikrokroki o stałej wielkości.
- **Przykład 2** obnaża zachowanie adaptacyjnego solvera `rk8pd`. Gęstość punktów znacznie wzrasta w regionach, gdzie funkcja ulega szybkim zmianom (ostre zakręty). W gładkich obszarach algorytm stawia znacznie większe kroki, oszczędzając zasoby obliczeniowe przy jednoczesnym zachowaniu precyzji [2].

3.2 Część B: Metoda Rungego-Kutty vs. Metoda Eulera

Rysunek 5 przedstawia numeryczne rozwiązania równania pierwszego rzędu w porównaniu z dokładną krzywą analityczną.

Graficzne porównanie wyraźnie pokazuje wyższą dokładność metody Rungego-Kutty 4. rzędu (RK4). Punkty danych RK4 idealnie układają się wzdłuż linii rozwiązania dokładnego. Z kolei metoda Eulera w sposób widoczny odbiega od prawdziwej krzywej, przestrzelając (*overshooting*) szczyty i doliny oscylacji.

Porównanie wydajności (*Efficiency Comparison*): Metoda Eulera jest podejściem 1. rzędu, w którym globalny błąd obcięcia (*global truncation error*) jest proporcjonalny do wielkości kroku ($O(h)$). Aby osiągnąć ten sam poziom dokładności co metoda RK4 ($O(h^4)$), metoda Eulera wymagałaby wykładniczo więcej kroków, co czyni ją wysoce nieefektywną w złożonych obliczeniach naukowych [2]. RK4 wymaga czterech ewaluacji funkcji na krok, ale pozwala na zastosowanie znacznie większych rozmiarów kroków, co ostatecznie czyni ją o wiele bardziej wydajną.

4 Conclusion

Zadania laboratoryjne podkreśliły znaczenie wyboru odpowiedniej metody numerycznej oraz strategii krokowania (*stepping strategy*) przy rozwiązywaniu równań różniczkowych zwyczajnych. Dla równań sztywnych (*stiff equations*) lub wykazujących szybkie fluktuacje, takich jak oscylator van der Pola, solwery z adaptacyjnym rozmiarem kroku (np. `rk8pd`) zapewniają najlepszy kompromis pomiędzy precyzją a wydajnością, automatycznie dostosowując rozdzielczość tam, gdzie jest to potrzebne [1]. Co więcej, porównanie w Części B potwierdziło empirycznie, że metody wyższego rzędu, takie jak RK4, są znacznie dokładniejsze i bardziej wydajne niż podstawowa metoda Eulera dla tej samej liczby kroków całkowania.

Bibliography

- [1] GNU Project: GNU Scientific Library Reference Manual: Ordinary Differential Equations (2024), <https://www.gnu.org/software/gsl/doc/html/ode-initval.html>, accessed strictly for GSL ODE routines
- [2] Heath, M.T.: Scientific computing: An introductory survey, chapter 9 - initial value problems for ordinary differential equations. Department of Computer Science, University of Illinois at Urbana-Champaign (2002), http://artemis.wszib.edu.pl/~funika/mois/lab11/heath_ODE.pdf

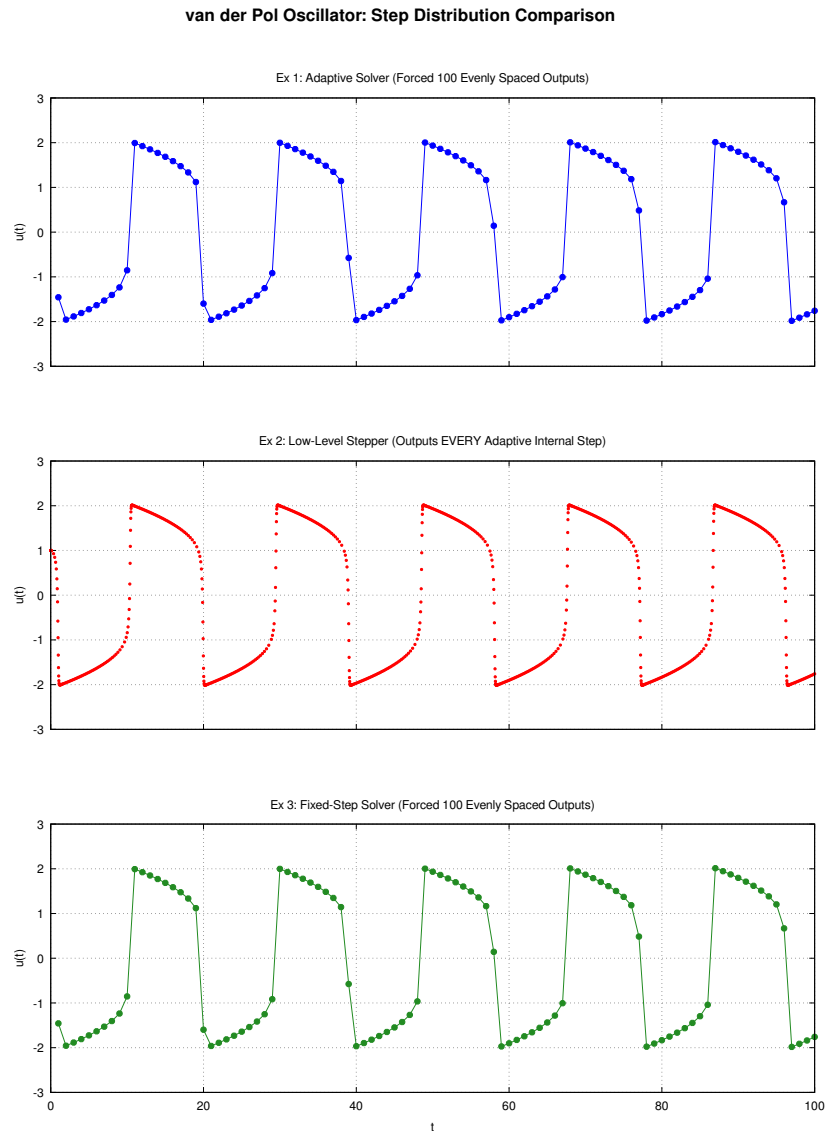


Figure 1: Porównanie rozkładu kroków dla oscylatora van der Pola z użyciem różnych solverów ODE z biblioteki GSL.

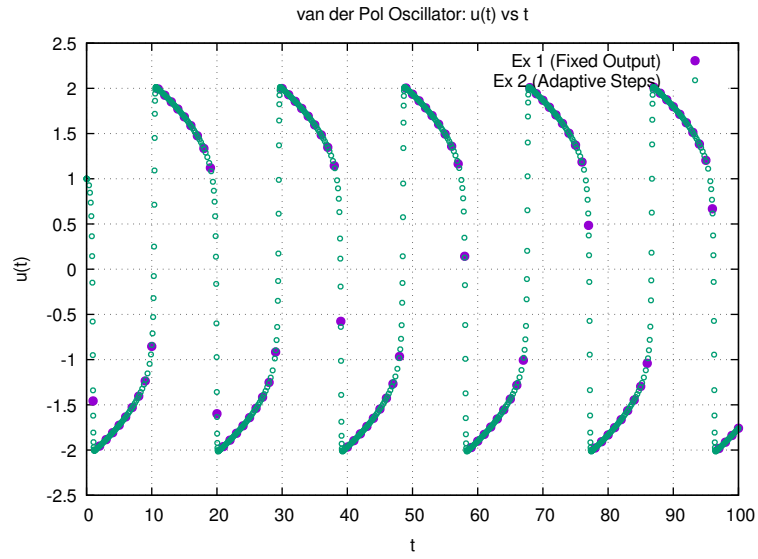


Figure 2: Nałożenie Przykładu 1 (Fixed Output) oraz Przykładu 2 (Adaptive Steps).

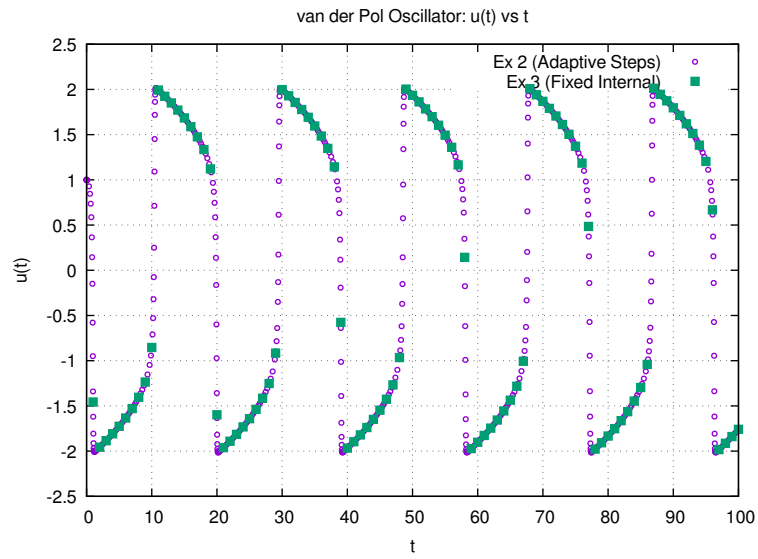


Figure 3: Nałożenie Przykładu 2 (Adaptive Steps) oraz Przykładu 3 (Fixed Internal Steps).

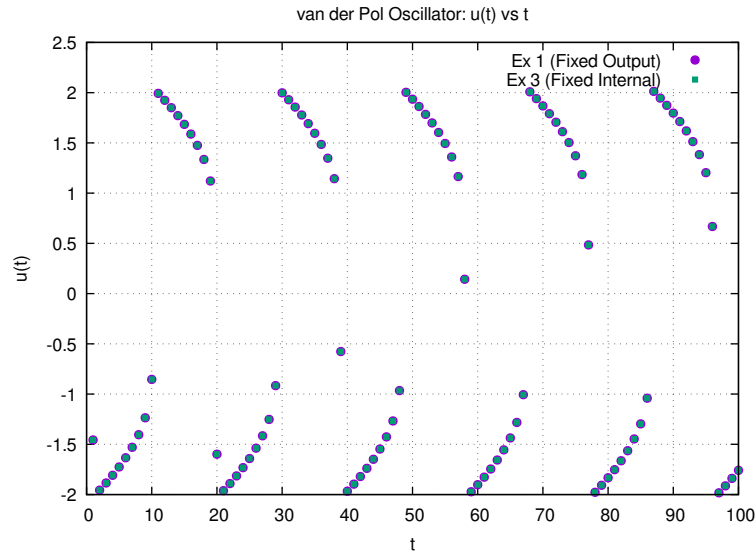


Figure 4: Nałożenie Przykładu 1 (Fixed Output) oraz Przykładu 3 (Fixed Internal Steps).

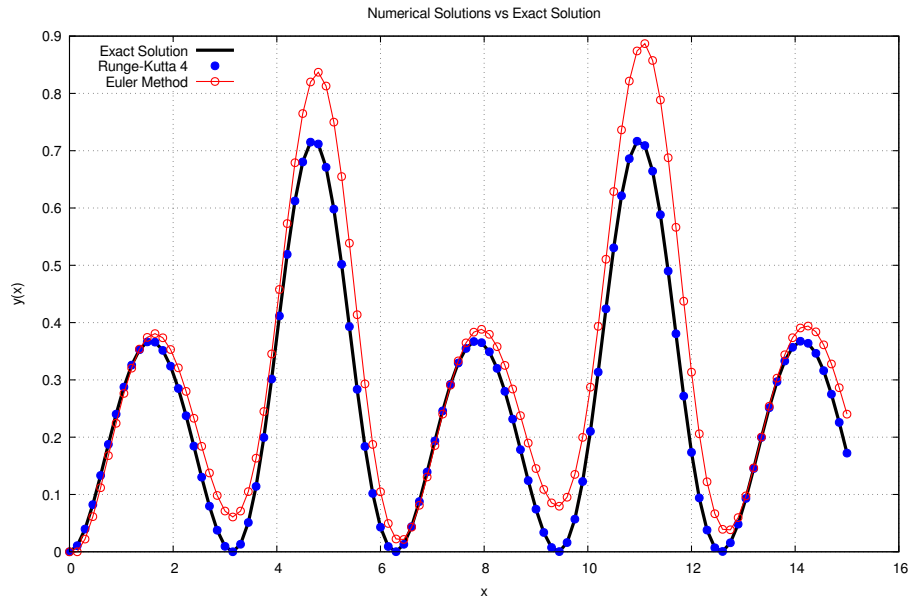


Figure 5: Rozwiązania numeryczne vs. Rozwiązanie dokładne dla $y' = \sin(x)\cos(x) - y\cos(x)$.